Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #8

Fail Safe: Fault Detection, an Emergency State, and the Watchdog

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 24, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Detect a fault:** run a highest-priority safety monitor that compares the commanded gear angle against the measured one and raises a fault only when they disagree *persistently*.
- ✓ **Model a fail-safe state:** extend the `FlightPhase` automaton with an `EMERGENCY` state reachable from *every* phase, which forces the gear to a safe position — a guarantee built into the model's structure.
- ✓ **Use the hardware watchdog:** subscribe the control task to the ESP-IDF Task Watchdog Timer so a hung controller is caught and reset instead of failing silently.
- ✓ **Prioritize safety:** justify why the monitor runs above the control task and prove that a detected fault overrides normal operation, driving the gear down and flashing the red LED.

2 Introduction

A controller that works only when everything goes right is not a critical system. The defining question of functional safety — the mindset behind DO-178C and ISO 26262 — is what the system does when something goes *wrong*: a stuck actuator, a lost sensor, a frozen task. This practice adds that layer to the landing gear. A safety monitor watches whether the gear actually reaches the position it was commanded; if it does not, an `EMERGENCY` state built into the flight-phase model forces the gear to a safe position and raises an alarm; and a hardware watchdog stands guard over the control task itself. The result is a system that fails safe by design, not by luck — the property that separates a demo from a controller you could defend in a certification review.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	`FlightPhase` model, extended here
2	ESP32 DevKit V1	1	Environment from Practice 2/3
3	USB-A to Micro-USB cable	1	Data cable (console)
4	ESP-IDF (v5.x)	1	FreeRTOS + Task Watchdog built in
5	Generated `scade_gen`	1	`FlightPhase` v2 (with `EMERGENCY`)
6	SG90 micro servo	1	Landing gear, GPIO 13, external 5 V
7	Potentiometer 10 kΩ	1	Measured gear position, GPIO 34
8	Three LEDs + 220 Ω	3	Phase + red alarm (GPIO 25/26/27)
9	External 5 V supply	1	For the servo (Practice 7)

Same hardware as Practice 7

No new components — this practice is almost entirely software and model. The potentiometer, which stood for the gear's measured position in Practice 7, is now the tool used to *simulate a stuck gear*: hold it away from the commanded angle and the safety monitor will detect the disagreement.

4 Theory & Block Reference

Safety splits cleanly into two responsibilities, and this practice keeps them in the right places: *detection* (deciding a fault has occurred) lives in the glue, where the sensors are; *reaction* (what safe thing to do about it) lives in the model, where it can be verified. A hardware watchdog backs both up as a last resort.

4.1. Detection vs. reaction — why the split matters

A safety monitor in the glue can read the raw sensors and time how long a disagreement lasts, which is a platform concern. But *what* the aircraft does in an emergency — force the gear down, refuse to retract — is control logic that must be reviewable and testable, so it belongs in the SCADE model as an explicit state. Putting the reaction in an `EMERGENCY` state means the safe behaviour is guaranteed by the automaton's structure, not by an `if` a reviewer has to trust.

Concept: `Fault detection with persistence`

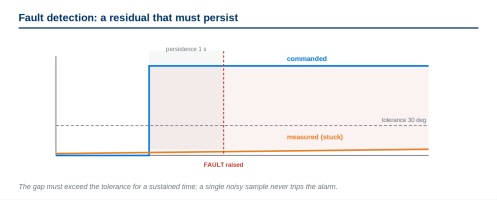

Figure 1: Commanded vs. measured angle; a fault is raised only after the gap persists — annotated view.

Key elements:

- **Residual:** the difference $|\text{cmd} - \text{measured}|$ each cycle.
- **Threshold:** a tolerance (e.g. 30°) that ignores normal servo travel and sensor noise.
- **Persistence:** the gap must exceed the threshold for a sustained time (e.g. 1 s) before a fault is declared — a debounce that rejects transients.

Behavior:

A single noisy sample must never trip the alarm; a genuinely stuck gear will hold the gap open and trip it. The monitor runs at the *highest* priority so detection is never delayed by the control or I/O tasks.

Concept: The EMERGENCY state (SCADE automaton)

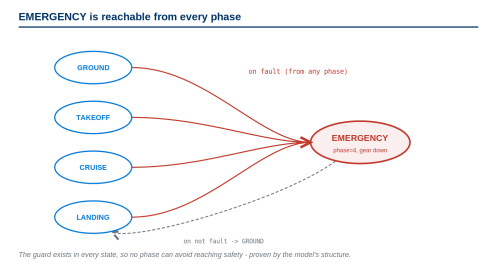

Figure 2: `EMERGENCY` reachable from every phase on `fault`, returning to `GROUND` when it clears — annotated view.

Key elements:

- **New input `fault`:** a Boolean from the safety monitor.
- **High-priority guard:** from *every* normal state, `fault` takes precedence over `next` and enters `EMERGENCY`.
- **Output `phase = 4`:** the glue maps it to gear-down (safe) and a flashing red LED.
- **Recovery:** `EMERGENCY`→`GROUND` when `fault` clears (or latched — see the Bonus).

Behavior:
Because the guard exists in every state, there is no phase from which the aircraft *cannot* reach safety — a property proven by the model, not argued in a comment.

Concept: Task Watchdog Timer (esp_task_wdt)

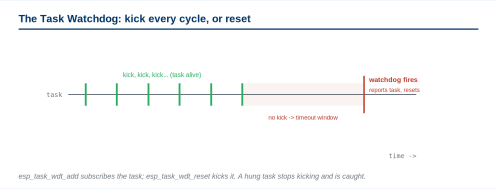

Figure 3: The control task must "kick" the watchdog each cycle or the chip resets — annotated view.

Key elements:

- **`esp_task_wdt_add(NULL)`:** subscribes the calling task to the watchdog.
- **`esp_task_wdt_reset()`:** "kicks" it each cycle to say the task is alive.
- **Timeout:** if no kick arrives within the window, the watchdog logs the offending task and resets the chip.

Behavior:
The watchdog catches the failure the software monitor cannot — the control task itself hanging. A reset is a blunt last resort, but a controller that reboots to a known safe state beats one frozen forever.

5 Worked Example

5.1. Objective

See the Task Watchdog fire, in isolation, before wiring it into the gear. A task kicks the watchdog for a few seconds, then deliberately stops; the watchdog reports the stuck task and resets the board. About 20 minutes.

5.2. Step-by-Step

1. **Create the project.**
   Copy the Practice 7 project (it carries the gear component and drivers):

```
cd C:\esp
xcopy /e /i lg_gear lg_safe
cd lg_safe
idf.py fullclean
```

2. **Write the watchdog demo.**
   Replace `main.c`:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_task_wdt.h"
#include "esp_log.h"
static const char *TAG = "SAFE";

static void worker(void *arg)
{
    esp_task_wdt_add(NULL);              // subscribe this task
    for (int i = 0; ; i++) {
        if (i < 6) {
            esp_task_wdt_reset();        // kick: "I am alive"
            ESP_LOGI(TAG, "kick %d", i);
        } else {
            ESP_LOGW(TAG, "stopped kicking - watchdog will fire...");
            while (1) { }                // simulate a hung task
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void app_main(void)
{
    xTaskCreate(worker, "worker", 3072, NULL, 3, NULL);
}
```

3. **Build, flash, and watch it fire.**
   `idf.py -p COM8 flash monitor`. The task kicks the watchdog six times, then hangs; a few seconds later the monitor prints `Task watchdog got triggered` naming `worker`, and the chip resets and starts over.

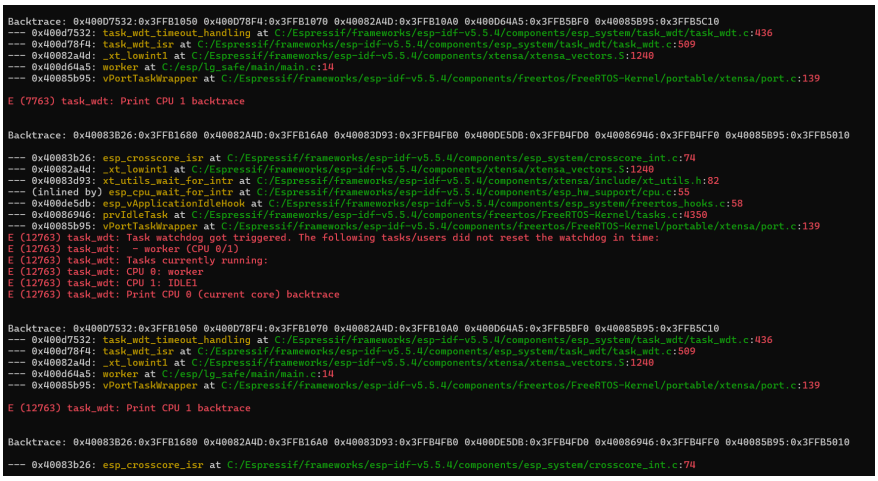

Figure 4: The watchdog reports the stuck `worker` task and resets the board.

Reading it right

The watchdog firing here is the *success* of the example: it proves the mechanism catches a task that stops responding. In the Main Activity the control task kicks it every cycle, so it never fires unless something is genuinely wrong.

6 Main Activity

6.1. Description

Make the landing gear fail safe. Extend the `FlightPhase` model with an `EMERGENCY` state entered from any phase on a `fault` input. In the glue, a highest-priority `safety_task` compares the commanded gear angle with the measured potentiometer angle and raises `fault` when they disagree for more than one second. On `fault` the model enters `EMERGENCY`, the gear is forced down, and the red LED flashes; the control task also kicks the Task Watchdog every cycle. Simulate a stuck gear by holding the knob away from the commanded angle.

6.2. Functional Specifications

- **Spec 1:** `FlightPhase` v2 adds an input `fault` and a fifth state `EMERGENCY`; from every normal state `fault` takes priority over `next`; the model outputs `phase` = 0–4.
- **Spec 2:** A `safety_task` at the highest priority reads the potentiometer, computes the residual vs. the commanded angle, and sets `fault` only after the residual exceeds 30° for ≥1 s (persistence).
- **Spec 3:** On `EMERGENCY` the gear servo is commanded to 0° (down = safe) and the red LED flashes at ≈2 Hz; the console logs `EMERGENCY`.
- **Spec 4:** The `model_task` subscribes to the Task Watchdog and kicks it every 100 ms cycle; a hang triggers a reset.

- **Spec 5:** Clearing the disagreement (returning the knob near the command) clears `fault` and the model returns to `GROUND`; normal phase operation resumes.

6.3. Activity Steps

1. **Extend the model in SCADE.**
   Open the `FlightPhase` automaton from Practice 6. Add the input `fault` (`bool`) and a new state `EMERGENCY` that outputs `phase = 4`. From each of the four normal states draw a transition to `EMERGENCY` guarded by `fault`, given higher priority than the `next` transition. From `EMERGENCY`, draw `EMERGENCY`→`GROUND` guarded by `not fault`. Simulate: pulsing `fault` from any phase must jump to `EMERGENCY` and hold until it clears.

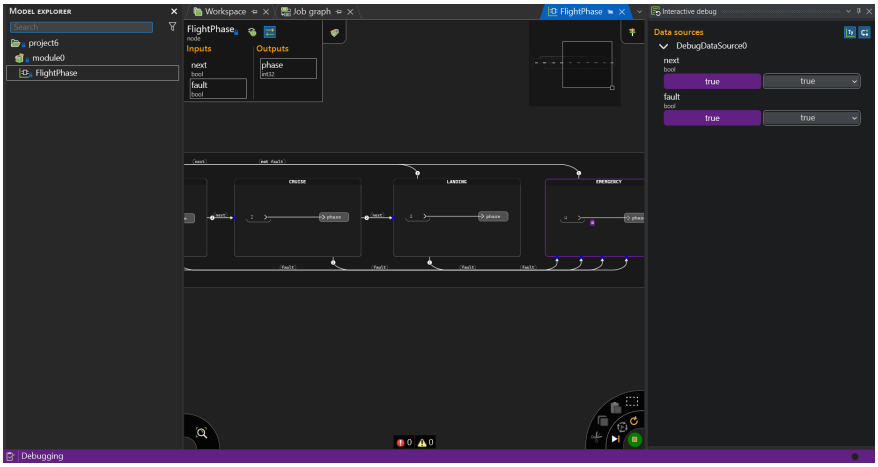

Figure 5: `FlightPhase` v2: `EMERGENCY` reachable from every phase on `fault`.

2. **Generate and integrate.**
   Run a Code Generation job and copy the new `scade_gen` into the project (Practice 3 workflow). The step signature now carries the extra input — confirm it in the header (e.g. `FlightPhase_module0(next, fault, &phase, &ctx)`).

3. **Write the safety-aware glue.**
   Replace `main.c` (servo/ADC helpers unchanged from Practice 7):

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "driver/ledc.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_task_wdt.h"
#include "esp_log.h"
#include "FlightPhase_module0.h"   // generated v2 - confirm name/signature

#define BTN 0
#define LED_GRN 25
#define LED_YEL 26
#define LED_RED 27
#define SERVO_GPIO 13
#define POT_CH  ADC_CHANNEL_6
#define TOL_DEG   30      // residual tolerance
#define FAULT_MS 1000     // persistence before declaring a fault
static const char *TAG = "SAFE";
```

```
static QueueHandle_t q_evt, q_act;
static SemaphoreHandle_t logmux;
static volatile int  gear_cmd  = 0;
static volatile bool fault_flag = false;

// --- servo helpers (Practice 7) ---
static void servo_init(void){ ledc_timer_config_t t={.speed_mode=LEDC_LOW_SPEED_MODE,
    .timer_num=LEDC_TIMER_0,.duty_resolution=LEDC_TIMER_16_BIT,.freq_hz=50,
    .clk_cfg=LEDC_AUTO_CLK}; ledc_timer_config(&t);
    ledc_channel_config_t c={.gpio_num=SERVO_GPIO,.speed_mode=LEDC_LOW_SPEED_MODE,
    .channel=LEDC_CHANNEL_0,.timer_sel=LEDC_TIMER_0,.duty=0,.hpoint=0};
    ledc_channel_config(&c); }
static void servo_write(int deg){ if(deg<0){deg=0;} if(deg>180){deg=180;}
    int us=500+(2500-500)*deg/180;
    uint32_t d=(uint32_t)((uint64_t)us*65535/20000);
    ledc_set_duty(LEDC_LOW_SPEED_MODE,LEDC_CHANNEL_0,d);
    ledc_update_duty(LEDC_LOW_SPEED_MODE,LEDC_CHANNEL_0); }

// --- input: BOOT -> next events ---
static void input_task(void *arg){
    gpio_config_t io={.pin_bit_mask=1ULL<<BTN,.mode=GPIO_MODE_INPUT,
        .pull_up_en=GPIO_PULLUP_ENABLE}; gpio_config(&io);
    int prev=1;
    while(1){ int now=gpio_get_level(BTN);
        if(prev==1&&now==0){ uint8_t e=1; xQueueSend(q_evt,&e,0); }
        prev=now; vTaskDelay(pdMS_TO_TICKS(20)); }
}

// --- SAFETY monitor: highest priority, detects a persistent mismatch ---
static void safety_task(void *arg){
    adc_oneshot_unit_handle_t adc;
    adc_oneshot_unit_init_cfg_t u={.unit_id=ADC_UNIT_1}; adc_oneshot_new_unit(&u,&adc);
    adc_oneshot_chan_cfg_t ch={.atten=ADC_ATTEN_DB_12,.bitwidth=ADC_BITWIDTH_DEFAULT};
    adc_oneshot_config_channel(adc,POT_CH,&ch);
    int bad=0;
    while(1){
        int raw; adc_oneshot_read(adc,POT_CH,&raw);
        int measured=raw*180/4095;
        int residual=measured-gear_cmd; if(residual<0){residual=-residual;}
        if(residual>TOL_DEG){ bad++; } else { bad=0; }
        fault_flag = (bad*100 >= FAULT_MS);   // 100 ms per sample
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// --- control task: FlightPhase v2 + watchdog + fail-safe outputs ---
static void model_task(void *arg){
    gpio_set_direction(LED_GRN,GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YEL,GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_RED,GPIO_MODE_OUTPUT);
    outC_FlightPhase_module0 ctx; swan_int32 phase;
    FlightPhase_reset_module0(&ctx);
    esp_task_wdt_add(NULL);                 // control task under the watchdog
    const int gear_of[5]={0,45,90,0,0};     // phase 4 = EMERGENCY -> down
    TickType_t last=xTaskGetTickCount(); int last_p=-1; bool blink=false;
    while(1){
        esp_task_wdt_reset();               // kick the watchdog
        uint8_t e; swan_bool next=false;
        if(xQueueReceive(q_evt,&e,0)==pdTRUE) next=true;
        swan_bool fault = fault_flag;
        FlightPhase_module0(next, fault, &phase, &ctx);
        int p=(int)phase;
        int cmd=gear_of[p]; gear_cmd=cmd;
        xQueueSend(q_act,&cmd,0);
        if(p==4){                           // EMERGENCY: flash red, others off
            gpio_set_level(LED_GRN,0); gpio_set_level(LED_YEL,0);
            blink=!blink; gpio_set_level(LED_RED,blink);
        } else {                            // normal phase LEDs
            gpio_set_level(LED_GRN,(p==0||p==1));
            gpio_set_level(LED_YEL,(p==1||p==2||p==3));
            gpio_set_level(LED_RED,0);
        }
```

```
        if(p!=last_p){
            const char *n[]={"GROUND","TAKEOFF","CRUISE","LANDING","EMERGENCY"};
            xSemaphoreTake(logmux,portMAX_DELAY);
            ESP_LOGW(TAG,"phase=%s gear_cmd=%d deg",n[p],cmd);
            xSemaphoreGive(logmux); last_p=p;
        }
        vTaskDelayUntil(&last,pdMS_TO_TICKS(100));
    }
}

// --- actuator ---
static void actuator_task(void *arg){
    servo_init(); int cmd;
    while(1) if(xQueueReceive(q_act,&cmd,portMAX_DELAY)==pdTRUE) servo_write(cmd);
}

void app_main(void){
    q_evt  = xQueueCreate(4,sizeof(uint8_t));
    q_act  = xQueueCreate(4,sizeof(int));
    logmux = xSemaphoreCreateMutex();
    xTaskCreate(safety_task,  "safety",3072,NULL,4,NULL);  // highest
    xTaskCreate(model_task,   "model", 3072,NULL,3,NULL);
    xTaskCreate(actuator_task,"act",   3072,NULL,2,NULL);
    xTaskCreate(input_task,   "input", 2048,NULL,2,NULL);
}
```

4. **Set dependencies and flash.**

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES scade_gen esp_driver_gpio
                                esp_driver_ledc esp_adc)
```

`idf.py -p COM8 flash monitor`. Advance to `CRUISE` (gear up, 90°), then *hold the knob near 0°* to simulate a gear stuck down. After one second the system enters `EMERGENCY`: the servo drives to 0°, the red LED flashes, and the console logs `EMERGENCY`. Return the knob near the command to recover.

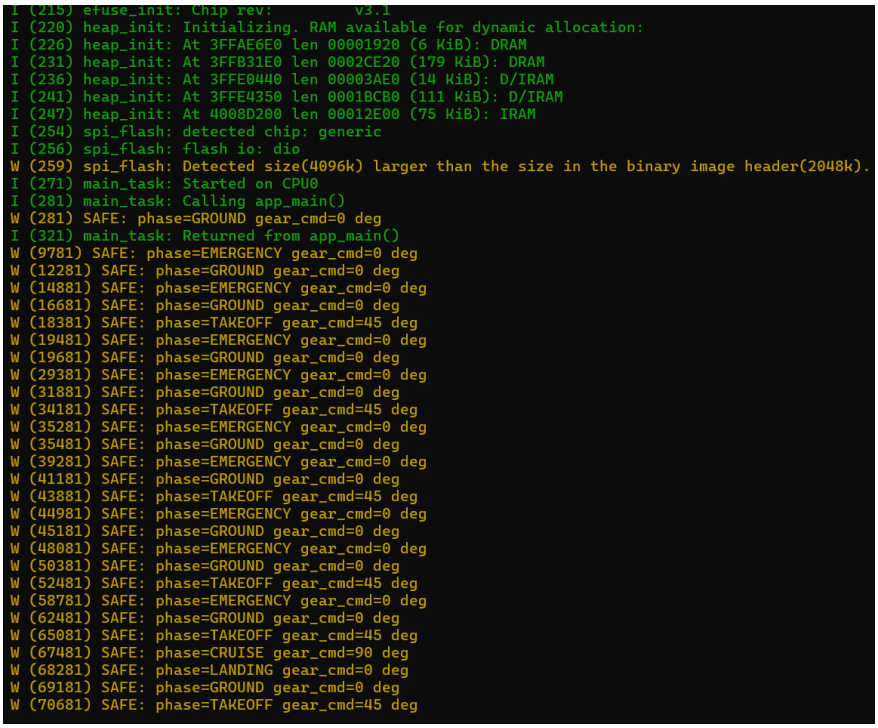

Figure 6: Console entering `EMERGENCY` after a sustained command/measured disagreement, then recovering.

7 Expected Results

Expected Results

- **SCADE Simulation:** Pulsing `fault` from any phase jumps to `EMERGENCY` (`phase = 4`) and holds until `fault` clears, then returns to `GROUND`. No phase can bypass the emergency guard.
- **Console / UART Output:** Normal phase lines, then an emergency after a sustained disagreement:

  ```
  W (2600) SAFE: phase=CRUISE gear_cmd=90 deg
  W (4100) SAFE: phase=EMERGENCY gear_cmd=0 deg
  W (7300) SAFE: phase=GROUND gear_cmd=0 deg
  ```

- **Physical Behavior:** Holding the knob far from the commanded angle for one second drives the gear servo down and flashes the red LED; the phase LEDs go dark. Returning the knob near the command recovers to normal operation. If the control task is ever made to hang, the watchdog resets the board (Worked Example).

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] Your `safety_task` requires the residual to exceed 30° for a full second before raising `fault`. Explain what would go wrong with a single-sample test (no persistence), and separately what a threshold of 0° (no tolerance) would do, in terms of servo travel time and ADC noise.

Q2. [*Analysis*] The `safety_task` runs at priority 4, above the model (3) and actuator (2). Trace what could happen to fault detection latency if it ran at the *lowest* priority instead, and explain why a safety monitor must not be starvable by normal work.

Q3. [*Analysis*] The emergency *reaction* lives in the SCADE model (a state) while the emergency *detection* lives in the glue (the monitor). Justify this division: what does putting the `EMERGENCY` state in the model let a reviewer verify that an `if` in the glue would not?

Q4. [*Synthesis*] Practice 9 controls four surfaces plus the gear. Describe how the `EMERGENCY` state should command *each* actuator to its own safe position (e.g. surfaces to neutral, gear down), and whether one shared `fault` or several independent faults is the better design. Justify.

Q5. [*Critical*] The Task Watchdog's response is to reset the whole chip. Argue whether a reset is an acceptable failure response for a landing-gear controller, compare it with the latched `EMERGENCY` state, and describe a situation where a reset would be *worse* than staying in a controlled emergency.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to model-based design for critical systems — in particular, what it means that the safe reaction is guaranteed by an `EMERGENCY` *state in the model rather than by defensive code, and how detection, reaction, and the watchdog together give a system that fails safe by design.*

10 Bonus Challenge

★ Bonus Challenge

Challenge:
Make the emergency *latched*, as a real system would: once `EMERGENCY` is entered, it must stay there even after the fault clears, until a deliberate reset — a long press of the `BOOT` button. Add a `reset` input to the automaton and change the `EMERGENCY`→`GROUND` guard to `reset` instead of `not fault`. Explain why a latched fault is safer than auto-recovery for a landing gear.

Hint:
A momentary sensor glitch should not silently un-declare an emergency. In the glue, detect a long press (level held low for >1 s) and feed it as `reset`*; keep the fault latched in the model so recovery is always a conscious act.*

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: The watchdog fires on your own control task

Symptom: `Task watchdog got triggered` names `model`, and the board resets during normal operation.

Cause: The control task subscribed with `esp_task_wdt_add` but a code path skips `esp_task_wdt_reset()` — often an early `continue` or a long blocking call inside the loop.

Fix: Kick the watchdog exactly once at the top of every loop iteration, before any branch, as in the listing.

Issue: EMERGENCY never triggers

Symptom: Holding the knob away from the command does nothing; the phase never reaches `EMERGENCY`.

Cause: The `fault` flag is not reaching the model (step called without the new argument), or the persistence counter is wrong (comparing samples instead of milliseconds), or the tolerance is too large.

Fix: Confirm the regenerated step signature takes `fault`; verify `bad*100 >= FAULT_MS`; test with the knob fully opposite the command.

Issue: EMERGENCY chatters on and off

Symptom: The red LED and phase flicker between `EMERGENCY` and a normal phase near the tolerance edge.

Cause: No hysteresis — the residual hovers right at the threshold, so the fault sets and clears every cycle.

Fix: Use separate set/clear thresholds (e.g. raise at 30°, clear only below 15°), or latch the emergency (Bonus). Persistence alone does not stop edge chatter.

Issue: Compile fails on `esp_task_wdt.h` or the new step argument

Symptom: Missing watchdog header, or a too-few-arguments error on `FlightPhase_module0`.

Cause: `esp_task_wdt.h` lives in `esp_system` (part of the core, no REQUIRES needed); the argument error means the glue still calls the Practice 6 signature without `fault`.

Fix: Include `esp_task_wdt.h`; update the call to `FlightPhase_module0(next, fault, &phase, &ctx)` to match the regenerated header.